



PROJECT DOCUMENTATION: EXAM MANAGEMENT SYSTEM

DATABASE PROJECT



2025

BY : HAJER EID

ENG. AHMAD WALED

INSTANT - BACKEND ROUND#34

0.1 Table of Contents

0.1 Table of Contents	1
0.2 Table of Codes.....	2
0.3 Table of Figures.....	2
0.4 Table of Tables	2
1. Executive Summary.....	3
1.1. Objective	3
1.2. Problem Statement.....	3
1.3. Proposed Solution.....	3
1.4. Key Achievements.....	3
2. Project Planning & Requirements	4
2.1. Functional Requirements.....	4
2.2. Non-Functional Requirements.....	4
2.2.1 Performance:	4
2.2.2 Data Integrity:	4
3. Database Design	5
3.1. Entity–Relationship (ER) Overview.....	5
3.1.1. simple DRD:	5
3.1.2 : details ERD	6
3.2. Logical Data Model.....	6
3.2.1. Defining the structure of the main tables	6
3.2.2. Mapping.....	7
4. Implementation & System Logic.....	8
4.1. Data Integrity Enforcement	8
4.1.1 Trigger: TR_PreventUserDeletion	8
4.2. Stored Procedures.....	9
4.2.1. SP_CreateNewExam.....	9
4.2.2. SP_SubmitAnswers	10
4.2.3. SP_GrantExamAccess	12
4.3. User-Defined Functions (UDFs)	13
4.3.1. FN_CalculateAttemptScore	13
4.3.2 FN_CheckPassingStatus.....	14
5. Performance Optimization & Reporting.....	16

5.1. Optimization Techniques.....	16
5.2. Reporting and Analytics	16
5.2.1. <i>VW_StudentExamResults</i>	16
5.2.2. <i>VW_InstructorCourseExams</i>	17
6. Conclusion & Future Recommendations.....	18
6.1. Summary of Achievements	18
6.2. Challenges and Lessons Learned.....	18
6.3. Future Enhancements	18
7. Final Remark	18

0.2 Table of Codes

1 code of TR_PreventUserDeletion	9
2 code of SP_CreateNewExam.....	10
3 code of SP_SubmitAnswers	12
4 code of SP_GrantExamAccess.....	13
5 code of FN_CalculateAttemptScore.....	14
6 code of FN_CheckPassingStatus	15
7 code of VW_StudentExamResults	17
8 code of VW_InstructorCourseExams	17

0.3 Table of Figures

Figure 1 :ERD 1	5
Figure 2 : ERD 2	6
Figure 3 : Mapping.....	7

0.4 Table of Tables

Table 1 : ROLES	4
Table 2 : structure	7

1. Executive Summary

1.1. Objective

To design and implement a **fully automated Exam Management System** utilizing **SQL Server** as the core database engine.

The system optimizes the entire examination lifecycle - from **random question generation** to **real-time automated grading** and **intelligent performance reporting** - minimizing manual effort and human error.

1.2. Problem Statement

Traditional manual examination systems are inefficient and prone to error due to the following issues:

- **Lack of consistency and fairness** due to non-randomized question distribution.
- **Time-intensive grading processes** that delay result publication.
- **Data inconsistency and weak referential integrity** leading to unreliable records.

These issues collectively reduce operational efficiency and scalability for academic institutions.

1.3. Proposed Solution

The project introduces a **robust relational database solution** powered by **SQL Server**, utilizing:

- **Stored Procedures** to encapsulate and securely execute business logic.
- **Triggers** to enforce data integrity and prevent unauthorized or unsafe operations.
- **User-Defined Functions (UDFs)** to modularize and centralize critical operations such as scoring and result validation.

This architecture ensures **accuracy**, **performance**, and **extensibility**, establishing a foundation for seamless integration with future **web** or **desktop** interfaces.

1.4. Key Achievements

- Achieved **instant automated grading** for objective-type questions through FN_CalculateAttemptScore.
- Enforced **secure data transactions** and referential integrity via triggers such as TR_PreventUserDeletion.
- Optimized **query performance and indexing**, resulting in fast and reliable data operations.
- Developed **dynamic reporting and analytics** using SQL Views and custom functions.

2. Project Planning & Requirements

2.1. Functional Requirements

The system provides the following core functionalities:

- **Exam Creation:** Define exam parameters such as duration, total marks, and number of questions.
- **Randomized Question Allocation:** Automatically draw questions from the course-specific question bank (implemented in SP_CreateNewExam).
- **Fair Scoring Mechanism:** Distribute marks proportionally among all questions.
- **Access Control:** Grant exam access to individual students or entire cohorts (SP_GrantExamAccess).
- **Submission Management:** Record each student's attempt along with start and end timestamps.

2.2. Non-Functional Requirements

2.2.1 Performance:

- All core operations (grading, result generation, and reporting) must execute with minimal latency.
- Efficient **indexing** and **query optimization** ensure scalability for large datasets.

2.2.2 Data Integrity:

- Strong **referential integrity** enforced through foreign key constraints and triggers (TR_PreventUserDeletion).
- Prevents deletion or modification of critical linked entities (instructors, students, managers).

2.3. User Roles and Responsibilities

<i>Role</i>	Responsibilities	Key Objects / Procedures
<i>Training Manager</i>	Supervising Intakes, granting students access to exams, and monitoring overall results.	SP_GrantExamAccess, VW_InstructorCourseExams
<i>Instructor</i>	Creating exams, randomly distributing questions, and determining the publication time.	SP_CreateNewExam, VW_InstructorCourseExams
<i>Student</i>	Attempt and submit exams, recording final answers, and reviewing the result.	SP_SubmitAnswers, VW_StudentExamResults

Table 1 : ROLES

3. Database Design

3.1. Entity–Relationship (ER) Overview

The database architecture for the Exam Management System is built upon a cohesive relational model that ensures high data integrity and efficient retrieval. The design is centered around three primary segments. First, the **User Management** segment links all system users to their specific roles via the Roles table, and then to their specialized entities (Student or Instructor), with the latter being protected by the TR_PreventUserDeletion mechanism. Second, the **Content Management** segment links the Question_Pool to the Course from which questions are drawn randomly. A crucial many-to-many relationship exists between Exam and Question_Pool and is implemented by the linking table **Exam_Questions**, which records the specific **degree_weight** of each question within that exam instance. The final and most critical segment is **Transactions and Results**, where the student's attempt is tracked in **Student_Exam_Attempt**. This attempt entity then links to all responses in the **Student_Answers** table. This is the central transactional table, recording either the **selected_option_id** or the **answer_text**, and driving the automated grading and aggregation processes executed by the system's functions and procedures. This structure guarantees a logical flow of data, from exam creation and weighting to final score computation.

3.1.1. simple DRD:

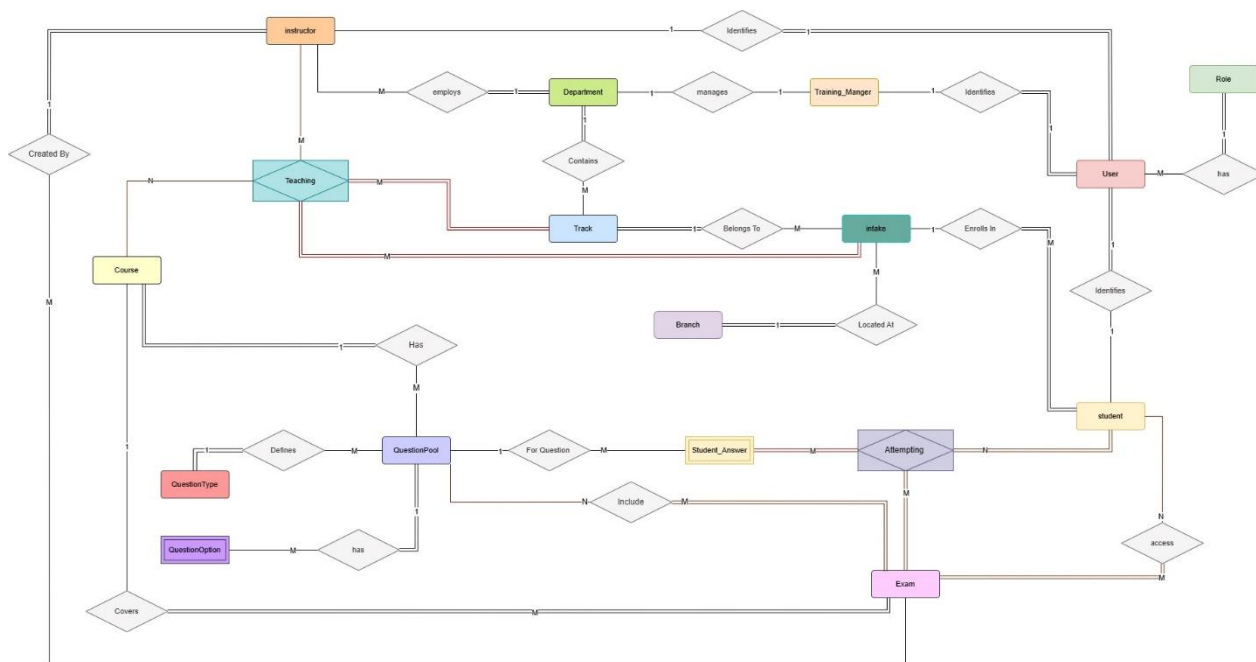


Figure 1 :ERD 1

3.1.2 : details ERD

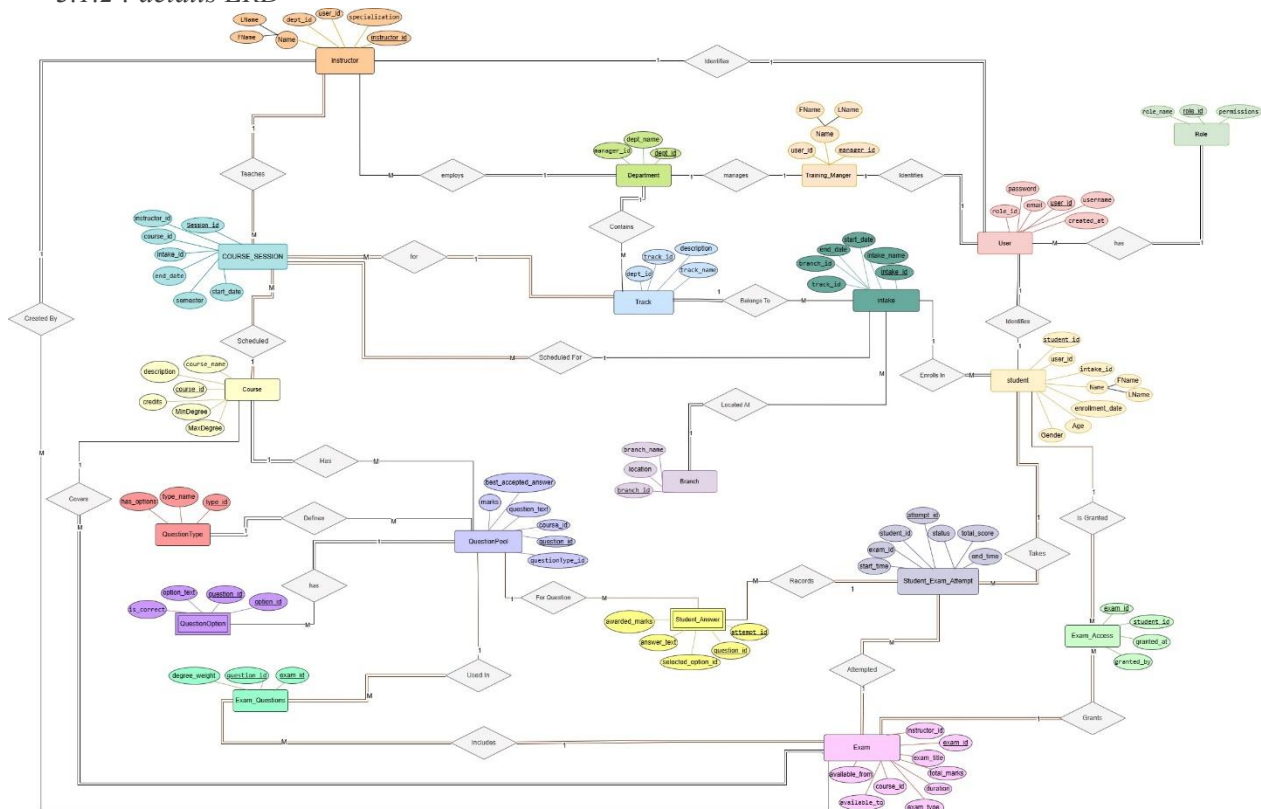


Figure 2 : ERD 2

3.2. Logical Data Model

Key design decisions include:

- **Normalized structure** (up to 3NF) to eliminate redundancy.
- **Composite primary keys** for multi-reference tables.
- **Cascading rules** carefully designed to prevent data loss.

This model ensures **data consistency**, **referential integrity**, and **high query efficiency**.

3.2.1. Defining the structure of the main tables

Table	Purpose	Primary keys (PK)	Foreign keys (FK*)
Users	User Identity & Role Management	user_id	role_id
Exam	Exam Configuration & Scheduling	exam_id	course_id , instructor_id
Question_Pool	Centralized Question Bank	question_id	course_id , question_type_id

4. Implementation & System Logic

4.1. Data Integrity Enforcement

4.1.1 Trigger: TR_PreventUserDeletion

- **Purpose:** Prevent the physical deletion of records from the **Users** table if those users have dependent foreign key relationships in critical tables (e.g., as an Instructor, Student, or Training Manager).
- **Mechanism:** This is implemented using an **INSTEAD OF DELETE** trigger. This design choice intercepts the original delete command and executes a conditional check. If dependencies are found, the system issues an error message and executes a **ROLLBACK TRANSACTION**, effectively canceling the deletion without raising a standard foreign key violation error.
- **Impact:** Guarantees **data consistency** and safeguards against the loss of critical user information. This mechanism ensures that records must be logically deactivated (disabled) rather than physically deleted, preserving the integrity of historical exam attempts and grading records.

```
USE ExamSystemDB;
GO

-- 1. TRIGGER > Prevent deletion of a user associated with a sub-entity (Instructor or manager)
CREATE OR ALTER TRIGGER TR_PreventUserDeletion
ON Users
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;

    -- Verify user association with > Train_Manager ?
    IF EXISTS (
        SELECT 1
        FROM deleted AS d
        INNER JOIN Train_Manager AS tm
        ON d.user_id = tm.user_id
    )
    BEGIN
        RAISERROR (N'Cannot delete a user who is currently assigned as a Training Manager.', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END

    -- Verify user association with > Instructor ?
    IF EXISTS (
        SELECT 1
        FROM deleted AS d
        INNER JOIN Instructor AS i
        ON d.user_id = i.user_id
    )
    BEGIN
        RAISERROR (N'Cannot delete a user who is currently registered as an Instructor.', 16, 1);
        ROLLBACK TRANSACTION;
    END
END
```

```

RETURN;
END

-- Verify user association with > Student ?
IF EXISTS (
    SELECT 1
    FROM deleted AS d
    INNER JOIN Student AS s
    ON d.user_id = s.user_id
)
BEGIN
    RAISERROR (N'Cannot delete a user who is currently registered as a Student.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END

-- If there is no association, Run the original deletion operation.
DELETE FROM Users
WHERE user_id IN (SELECT user_id FROM deleted);
END
GO

```

1 code of TR_PreventUserDeletion

4.2. Stored Procedures

4.2.1. SP_CreateNewExam

- **Purpose:** Automate exam creation with a balanced and randomized selection of questions.
- **Logic:**
 - This procedure is critical for ensuring fairness and consistency. It begins by validating that the desired number of questions is available in the Question_Pool for the specified course. It then calculates a proportional degree weight for each question based on the exam's total_marks. The core logic uses **ORDER BY NEWID()** to randomly select the required number of questions, inserting the definitive list into the **Exam_Questions** linking table. This final step guarantees that the exam content is fixed and prepared for student attempts.

```

USE ExamSystemDB;
GO

--Procedure: Create the exam & randomly draw questions.
CREATE OR ALTER PROCEDURE SP_CreateNewExam
    @CourseID INT,
    @InstructorID INT,
    @Title NVARCHAR(255),
    @ExamType VARCHAR(50), -- 'Exam' or 'Corrective'
    @Duration SMALLINT,
    @AvailableFrom DATETIME2,
    @AvailableTo DATETIME2,
    @TotalMarks DECIMAL(5,2),
    @NumQuestions INT

```

```

AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @NewExamID INT;
    DECLARE @MarkPerQuestion DECIMAL(5,2);

    -- 1. Ensure that the required questions are in the question bank.
    IF (
        SELECT COUNT(*)
        FROM Question_Pool
        WHERE course_id = @CourseID
    ) < @NumQuestions
    BEGIN
        RAISERROR (N'Insufficient number of questions available in the question pool for this course.', 16,
1);
        RETURN;
    END
    --Calculating grades
    SET @MarkPerQuestion = @TotalMarks / @NumQuestions;

    -- 2. Insert into the Exam Table
    INSERT INTO Exam (course_id, instructor_id, exam_title, exam_type, duration, total_marks,
available_from, available_to)
    VALUES
        (@CourseID, @InstructorID, @Title, @ExamType, @Duration, @TotalMarks, @AvailableFrom,
@AvailableTo);

    SET @NewExamID = SCOPE_IDENTITY();

    -- 3. Randomly pull questions and associate them with Exam_Questions:

    ---Linking Questions---
    -- This logic ensures that the pulled questions are only for the exam session.
    INSERT INTO Exam_Questions (exam_id, question_id, degree_weight)

    SELECT TOP (@NumQuestions)
        @NewExamID AS exam_id,
        qp.question_id,
        @MarkPerQuestion -- Distribute the grade equally
    FROM Question_Pool AS qp
    WHERE qp.course_id = @CourseID
    ORDER BY NEWID(); -- Random withdrawal order (generate id/row)

    SELECT @NewExamID AS CreatedExamID,
        N'Exam created successfully with ' + CAST(@NumQuestions AS NVARCHAR(10)) + N'
questions.' AS StatusMessage;
END
GO

```

2 code of SP_CreateNewExam

4.2.2. SP_SubmitAnswers

- **Purpose:** Record all student responses for a single attempt, compute the final score, and finalize the attempt status.

- **Logic:**
- This procedure manages the sensitive submission process. It receives the student's answers via a User-Defined Table Type (UDTT), inserts them into the **Student_Answers** transactional table, and updates the attempt's end_time and status to 'Submitted'. Crucially, it then executes the function **FN_CalculateAttemptScore** to compute the final grade. All these operations are wrapped within a **SQL Transaction** (ensuring **atomicity**), which guarantees that either all answers and the final score are successfully recorded, or none of them are.

```

USE ExamSystemDB;
GO

-- 1. create new TYPE {list of student Answer }
CREATE TYPE dbo.AnswerListType AS TABLE
(
    QuestionID INT NOT NULL,
    SelectedOptionID INT NULL, -- (MCQ/T&F)
    AnswerText NVARCHAR(MAX) NULL, -- (Essay)
    AwardedMarks DECIMAL(5, 2) NULL --for essay answers that may be corrected later by the instructor.
);
GO

-----
USE ExamSystemDB;
GO

-- 2. Procedure: To record answers and calculate the final grade
CREATE OR ALTER PROCEDURE SP_SubmitAnswers
    @AttemptID BIGINT,
    @StudentAnswers dbo.AnswerListType READONLY -- Pass the answer list as a table
AS
BEGIN
    SET NOCOUNT ON;
    -- Everything must be executed successfully or fail completely (Transaction) >
    BEGIN TRANSACTION;

    DECLARE @CalculatedScore DECIMAL(5, 2);

    -- 1. Insert all answers from "the passed list" into a "Student_Answers"
    --
    INSERT INTO Student_Answers (attempt_id, question_id, selected_option_id, answer_text,
    awarded_marks)
    SELECT
        @AttemptID,
        QuestionID,
        SelectedOptionID,
        AnswerText,
        AwardedMarks --It is sent from the Form (usually NULL or 0 at this stage)
    FROM
        @StudentAnswers;

    -- 2. Setting the time for the attempt to end
    DECLARE @EndTime DATETIME2 = GETDATE();

    -- 3. Update End time and attempt status

```

```

UPDATE Student_Exam_Attempt
SET
    end_time = @EndTime,
    status = N'Submitted'
WHERE
    attempt_id = @AttemptID;

-- 4. Calculate the final grade using the function :
SET @CalculatedScore = dbo.FN_CalculateAttemptScore(@AttemptID);

-- 5. Update the final overall grade in the Student_Exam_Attempt table
UPDATE Student_Exam_Attempt
SET
    total_score = @CalculatedScore
WHERE
    attempt_id = @AttemptID;

-- 6. End of TRANSACTION
IF @@ERROR <> 0
BEGIN
    ROLLBACK TRANSACTION;
    RAISERROR (N'An error occurred during submission. Transaction rolled back.', 16, 1);
    RETURN;
END

COMMIT TRANSACTION;

SELECT N'Exam submitted successfully. Calculated Score: ' + CAST(@CalculatedScore AS
NVARCHAR(10)) AS Status;
END
GO

```

3 code of SP_SubmitAnswers

4.2.3. SP_GrantExamAccess

- **Purpose:** Efficiently grant exam access permissions to an entire student cohort or intake at once.
- **Logic:** Designed for administrative efficiency, this procedure allows the Training Manager (or Instructor) to grant access using the IntakeID. It identifies all students belonging to the specified intake and inserts their IDs into the **Exam_Access** table along with the granted_at time and granted_by user. A vital integrity check is included using a **NOT EXISTS** clause, which prevents the accidental creation of **duplicate access records** for students who may have already been granted permission.

```

USE ExamSystemDB;
GO
--procedure: To grant access to the exam
CREATE OR ALTER PROCEDURE SP_GrantExamAccess
    @ExamID INT,
    @IntakeID INT,
    @GrantedBy INT
AS
BEGIN
    SET NOCOUNT ON;

```

```

-- Check the exam and Intake details
IF NOT EXISTS (SELECT 1 FROM Exam WHERE exam_id = @ExamID)
BEGIN
    RAISERROR (N'Exam ID not found.', 16, 1);
    RETURN;
END

--Insert in Exam_Access :all stu. associated with the Intake (@IntakeID)
INSERT INTO Exam_Access (exam_id, student_id, granted_at, granted_by)
SELECT
    @ExamID,
    S.student_id,
    GETDATE(),
    @GrantedBy -- instructor / manger ID
FROM
    Student S
WHERE
    S.intake_id = @IntakeID

-- Preventing the same student from get access to the same exam multiple times.
AND NOT EXISTS (
    SELECT 1 FROM Exam_Access EA
    WHERE EA.exam_id = @ExamID AND EA.student_id = S.student_id
);

SELECT CAST(@@ROWCOUNT AS NVARCHAR(10)) + N' students granted access to Exam ID ' +
CAST(@ExamID AS NVARCHAR(10)) AS StatusMessage;
END
GO

```

4 code of SP_GrantExamAccess

4.3. User-Defined Functions (UDFs)

4.3.1. FN_CalculateAttemptScore

- **Purpose:** Calculate each student's total exam score.
- **Logic:**
 - Uses **LEFT JOIN** to match answers with correct options and **CASE** expressions to award scores for objective questions. It then aggregates these results along with the **manually awarded marks** for essay questions, centralizing all complex scoring logic.

```

USE ExamSystemDB;
GO

--Function: To calculate the [total score] for a student's exam attempt
CREATE OR ALTER FUNCTION FN_CalculateAttemptScore (@AttemptID BIGINT)
RETURNS DECIMAL(5, 2)
AS
BEGIN
    DECLARE @TotalScore DECIMAL(5, 2) = 0.00;

    -- Calculating the score for <automatically graded> questions (MCQ, True/False)

```

```

SELECT @TotalScore = SUM(
  -- use the CASE > to check if the answer is correct.
  CASE
    -- 1. Check the correct answer (MCQ/T&F)
    WHEN T.has_options = 1
      AND SA.selected_option_id IS NOT NULL -- The selected answer must be present
      AND QOC.is_correct = 1 --check correct linked option
      THEN EQ.degree_weight -- Award full marks

    -- Case 2: Essay or objective questions where automatic grading failed
    --rely on the [grade manually] awarded by > instructor (awarded_marks).
    WHEN T.has_options = 0
      THEN SA.awarded_marks

    ELSE 0.00 -- other cases (wrong answer or question not answered automatically)
  END
)
FROM Student_Answers SA
INNER JOIN
  Question_Pool QP ON SA.question_id = QP.question_id
INNER JOIN
  QuestionType T ON QP.question_type_id = T.type_id
INNER JOIN
  Exam_Questions EQ ON SA.question_id = EQ.question_id

--Link with options table > to check correct option

LEFT JOIN ---the written Question has no option <LEFT>
  Question_Option QOC ON SA.question_id = QOC.question_id
  AND SA.selected_option_id = QOC.option_id --link stu. Answer with choice in QuestionPool

WHERE SA.attempt_id = @AttemptID;
RETURN @TotalScore;
END
GO

```

5 code of FN_CalculateAttemptScore

4.3.2 FN_CheckPassingStatus

- **Purpose:** Verify whether a student meets the minimum passing criteria.
- **Logic:**
 - Compares the earned score against the **MinDegree** threshold retrieved dynamically from the **Course** table, returning "Passed" or "Failed" accordingly.

```

USE ExamSystemDB;
GO

--function : to Comparing the exam score & min passing grade (course).
CREATE OR ALTER FUNCTION FN_CheckPassingStatus
(
  @Score DECIMAL(5, 2),
  @CourseID INT

```

```

)
RETURNS NVARCHAR(10)
AS
BEGIN
    DECLARE @MinDegree DECIMAL(5, 2);

    -- get the minimum passing grade from the course table.
    SELECT @MinDegree = MinDegree
    FROM Course
    WHERE course_id = @CourseID;

    IF @Score >= @MinDegree
        RETURN N'Passed';

    RETURN N'Failed';
END
GO

```

6 code of FN_CheckPassingStatus

5. Performance Optimization & Reporting

5.1. Optimization Techniques

- **Indexing:** Implemented non-clustered indexes strategically on frequently used columns, particularly those used in **JOIN** and **WHERE** clauses, to accelerate queries and reduce disk I/O.

Examples:

- **IX_StudentAnswers_Attempt** (crucial for enhancing grading performance)
- **IX_Attempt_StudentExam** (improves result retrieval speed).
- **Query Optimization:** Utilized execution plans and query statistics during development to minimize I/O overhead and execution time across all Stored Procedures and Views.

5.2. Reporting and Analytics

5.2.1. VW_StudentExamResults

- Provides a unified report displaying student information, **total scores**, and the final **pass/fail status**.
- **Integrates FN_CheckPassingStatus** for dynamic, real-time result validation, giving end-users immediate logical feedback.

```
USE ExamSystemDB;
GO

-- 2.displaying the detailed results of students' attempts.
CREATE VIEW VW_StudentExamResults
AS
SELECT
    ST.st_Fname + ' ' + ST.st_Lname AS Student_Name,
    C.course_name,
    E.exam_title,
    SEA.attempt_id,
    SEA.total_score AS Score_Achieved,
    E.total_marks AS Max_Score,
    -- Use the function to check the success status
    dbo.FN_CheckPassingStatus(SEA.total_score, C.course_id) AS Passing_Status,
    SEA.status AS Attempt_Status,
    SEA.end_time
FROM
    Student_Exam_Attempt SEA
INNER JOIN
    Exam E ON SEA.exam_id = E.exam_id
INNER JOIN
    Course C ON E.course_id = C.course_id
INNER JOIN
    Student ST ON SEA.student_id = ST.student_id
WHERE
    SEA.status = N'Submitted'; -- Show only submitted attempts
```

```
GO
```

```
-----  
--CHECK:
```

```
SELECT * FROM VW_StudentExamResults;
```

7 code of VW_StudentExamResults

5.2.2. *VW_InstructorCourseExams*

- Summarizes exam statistics for instructors, including **total attempts** and success metrics per exam.
- Enables instructors to monitor course and exam performance effectively using aggregated data.

```
USE ExamSystemDB;
```

```
GO
```

```
-- 1. display exam and course summaries (for Instructors/Managers)
```

```
CREATE OR ALTER VIEW VW_InstructorCourseExams
```

```
AS
```

```
SELECT
```

```
    E.exam_id,
```

```
    E.exam_title,
```

```
    C.course_name,
```

```
    E.total_marks,
```

```
    U_Ins.username AS Instructor_Name,
```

```
    -- Number of students who started Attempt
```

```
    COUNT(SEA.attempt_id) AS Total_Attempts,
```

```
    E.available_from,
```

```
    E.available_to
```

```
FROM
```

```
    Exam E
```

```
INNER JOIN
```

```
    Course C ON E.course_id = C.course_id
```

```
INNER JOIN
```

```
    Instructor I ON E.instructor_id = I.instructor_id
```

```
INNER JOIN
```

```
    Users U_Ins ON I.user_id = U_Ins.user_id
```

```
LEFT JOIN --to show even exams that no one has taken.
```

```
    Student_Exam_Attempt SEA ON E.exam_id = SEA.exam_id
```

```
GROUP BY
```

```
    E.exam_id,
```

```
    E.exam_title,
```

```
    C.course_name,
```

```
    E.total_marks,
```

```
    U_Ins.username,
```

```
    E.available_from,
```

```
    E.available_to;
```

```
GO
```

```
-----  
--CHECK:
```

```
SELECT * FROM VW_InstructorCourseExams;
```

8 code of VW_InstructorCourseExams

6. Conclusion & Future Recommendations

6.1. Summary of Achievements

The Exam Management System successfully delivers a **comprehensive, automated, and reliable solution** to academic institutions.

It achieves:

- **Full Cycle Automation:** Automated the entire process from **randomized creation** (SP_CreateNewExam) to **instantaneous grading** (FN_CalculateAttemptScore).
- **Zero Tolerance Data Integrity:** Eliminated manual errors by enforcing strict business rules via Triggers (TR_PreventUserDeletion) and Transactional Logic (SP_SubmitAnswers).
- **Optimized Scalability:** Demonstrated high performance and query efficiency suitable for multi-user environments through strategic **Indexing** and **View** creation.

6.2. Challenges and Lessons Learned

- **Complex Function Development:** The creation of FN_CalculateAttemptScore initially faced issues with subquery limitations in aggregate functions.
 - **Resolution:** Successfully implemented **JOIN-based methodologies** to compute aggregated results within the function, confirming expertise in managing complex SQL limitations.
- **Atomic Consistency:** Reinforced the understanding of **concurrency control** by guaranteeing that all submission operations (SP_SubmitAnswers) are atomic, securing results under heavy system load.

6.3. Future Enhancements

- **API Readiness:** Develop **SQL Server endpoints or web services** (e.g., using technologies like ASP.NET or Python) to abstract the Stored Procedures, facilitating seamless front-end integration.

7. Final Remark

This project establishes a **strong technical foundation** for scalable, secure, and intelligent examination management.

It demonstrates advanced database engineering principles, robust procedural logic, and a commitment to data integrity — **a complete transition from manual inefficiency to automated precision.**